

Functional Programming - 1|Lecture

- Intermediate: Functional Programming
- Lambda Functions
- Higher Order function
- Decorators



Functional Programming

↳ is a paradigm where computation is viewed as the evaluation of mathematical functions.



↳ Immutable Data :-
no changing state of mutable data is allowed.

↳ Declarative style :-
Instead of instructing "how" focus on "what".

lambda

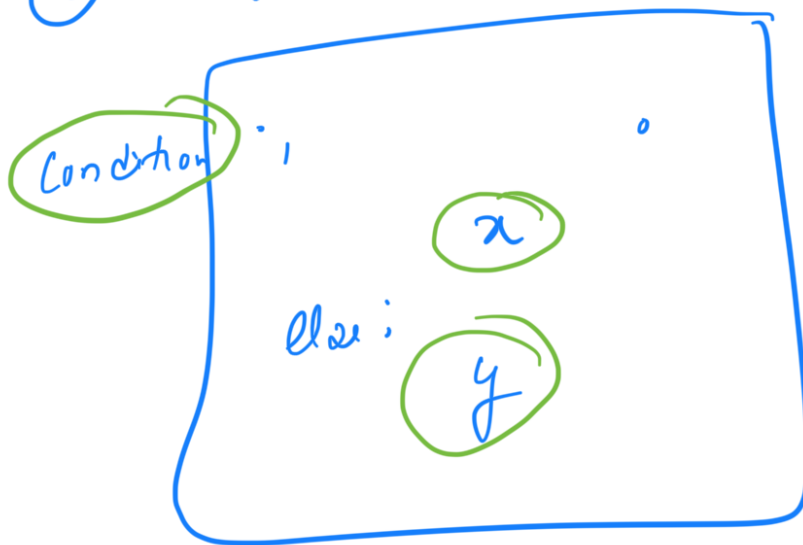
```
def square(x):  
    return x ** 2
```

```
✓ lambda x: x ** 2
```

$$\text{Square} = 1 \text{ u.}$$

Benefits

- ① Concise and Readable
- ② More maintainable
- ③ Efficient



$a = \lambda x : x \text{ if } x \geq 5 \text{ else } 0$

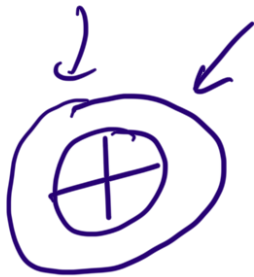
$a(7)$

Higher Order Function:-

- ① takes another function as input
- ② Return another function as output

fun(a, b)
↳ str

Advantages



- ① More reusable
- ② More concise
- ③ More flexible

Decorator :- It is a function that takes another function as input
↳ add some extra behavior
↳ returns a new function

#decorator functions

```
def pretty(func):  
    def inner():  
        print("-"*50)  
        func()  
        print("-"*50)
```

```
    return inner
```

```
def foo():
```

```
print("This is demo for pretty function")
```

```
def foo():
```

```
    print("This is demo for pretty function")
```

pretty
Print ()
func
Print ()
decorator

(a) pretty

```
def foo():  
    print("Hello")
```

def foo():
 print("Hello")

=

how = pretty(foo)
how()